Euclid's Theorem in Lean 4

Lean 4 · Mathlib4 · Verso

For every natural number `n` there is a prime `p` with `n < p`. Euler's form of Euclid's construction: put `m := n ! + 1` and `p := m.minFac`; then `p` is prime, `p ∣ m`, and `p` does not divide `n !` (else `p ∣ 1`), so `n < p`. The proof below is the file `EuclidPrimes.lean` verbatim, from `namespace` to `end`; it is re-elaborated here by Verso, so it is checked code, not a transcript. Its imports are `Mathlib.Data.Nat.{Factorial.Basic, Prime.Defs, Prime.Factorial, Prime.Infinite}`.

```
namespace EuclidPrimes

/-- `n! + 1` is never `1`, because `n!` is positive.
This is the side condition under which `Nat.minFac` is guaranteed to be prime. -/
theorem factorial_add_one_ne_one (n : ℕ) : n ! + 1 ≠ 1 := by
  intro heq
  have h₁ : n ! + 1 = 0 + 1 := by simpa using heq
  have h₂ : n ! = 0 := Nat.add_right_cancel h₁
  exact Nat.factorial_ne_zero n h₂

/-- A prime at most `n` divides `n!`, because `n!` is the product
`1 · 2 ⋯ n`, one of whose factors is `p`. -/
theorem prime_dvd_factorial {p n : ℕ} (hp : p.Prime) (hpn : p ≤ n) : p ∣ n ! :=
  Nat.dvd_factorial hp.pos hpn

/-- **Key lemma.** The least prime factor of `n! + 1` does not divide `n!`.

If it did, it would divide both `n! + 1` and `n!`, hence their difference `1`,
which no prime divides. -/
theorem minFac_factorial_add_one_not_dvd (n : ℕ) : ¬ (n ! + 1).minFac ∣ n ! := by
  intro hdvd
  have hp : (n ! + 1).minFac.Prime := Nat.minFac_prime (factorial_add_one_ne_one n)
  have hdvd' : (n ! + 1).minFac ∣ n ! + 1 := Nat.minFac_dvd _
  have hsub : (n ! + 1).minFac ∣ (n ! + 1) - n ! :=
    Nat.dvd_sub hdvd' hdvd
  have hone : (n ! + 1).minFac ∣ 1 := by simpa using hsub
  exact hp.ne_one (Nat.dvd_one.mp hone)

/-- The least prime factor of `n! + 1` is strictly larger than `n`. -/
theorem minFac_factorial_add_one_gt (n : ℕ) : n < (n ! + 1).minFac := by
  have hp : (n ! + 1).minFac.Prime := Nat.minFac_prime (factorial_add_one_ne_one n)
  have hle : ¬ (n ! + 1).minFac ≤ n := fun h =>
    minFac_factorial_add_one_not_dvd n (prime_dvd_factorial hp h)
  exact Nat.lt_of_not_le hle

/-- **Euclid's theorem**, by Euler's construction: for every `n` there is a prime
`p` with `n < p`. -/
theorem exists_prime_gt (n : ℕ) : ∃ p, Nat.Prime p ∧ n < p :=
  ⟨(n ! + 1).minFac, Nat.minFac_prime (factorial_add_one_ne_one n),
    minFac_factorial_add_one_gt n⟩

/-- The statement of the task, quantified explicitly. -/
theorem forall_exists_prime_gt : ∀ n : ℕ, ∃ p, Nat.Prime p ∧ n < p := exists_prime_gt

/-- Our result, restated in the shape Mathlib uses (`≤` instead of `<`). -/
theorem exists_prime_ge (n : ℕ) : ∃ p, n ≤ p ∧ Nat.Prime p :=
  let ⟨p, hp, hlt⟩ := exists_prime_gt n
  ⟨p, Nat.le_of_lt hlt, hp⟩

/-- Mathlib's own `Nat.exists_infinite_primes`, for comparison: it states that for
every `n` some prime is `≥ n`, which is the same fact about the infinitude of primes. -/
theorem agrees_with_mathlib (n : ℕ) : ∃ p, n ≤ p ∧ Nat.Prime p :=
  Nat.exists_infinite_primes n

end EuclidPrimes
```

`#print axioms EuclidPrimes.exists_prime_gt` gives `[propext, Classical.choice, Quot.sound]`, and `lake build` succeeds with no `sorry` and no `admit`.